

Agent Harness Review

增量研究 - 未完成页面视觉验收

研究修订稿

资料截面

2026-09-19

Agent Harness 增量研究：哪些变化值得改进这本书

核验日期：2026-09-19。书稿基线：aled264462d9b61c260c1dc417243303bfc60b66。正文资料截面为 2026-08-27/28，上一轮表达改写发布于 2026-09-10。

Executive Summary - 结论

这段时间确实有值得纳入的变化，而且不只是又出现了几个 Agent 产品。最直接影响企业平台选型的，是托管 Harness 与自管执行环境开始分开提供：OpenAI 的 Agents API 已进入公测，Cursor 能把执行机器放到企业网络内，微软把可组合的 Harness 作为框架能力公开说明。平台现在要分别决定谁运行模型循环、谁运行工具、谁保存会话和证据，不能只问“买还是自研”。[1][2][5][8]

自我进化研究也出现了更扎实的分工：HarnessDev 评价模型能否造出可运行的 Harness；HarnessEvolve 用参考轨迹辅助诊断；JIT-Agent 训练一个生成 Harness 的辅助模型；SoL-Pi 把省钱作为搜索目标。这些研究同时暴露了边界：改进不总能迁移到另一模型，参考轨迹需要监督，降低 token 可能牺牲得分，单条演化轨迹不能证明长期可靠性。[9][11][12][23]

这些材料总体支持本书重视执行、反馈和验证的方向，但要求降低几类过满的表述：复杂架构不必然优于简单循环；子代理不必然省钱；外置验证不等于验证独立；自托管沙箱不等于数据不出企业边界。对原书的修改应围绕这些具体条件展开，而不是追着产品名补一串新章节。[2][7][9][13]

这份增量研究与本轮逐章审阅、案例运行报告配套使用。它没有把读过论文当作复现实验，也没有把社区故障报告当作确认漏洞。新增资料应先进入证据账本，再用于重写受影响段落；原书已确认的术语、协议和安全伪代码问题，优先级高于补充前沿材料。

Introduction - 范围与方法

本轮优先检索 8 月 28 日之后至 9 月 19 日的更新，并回查以前登记或遗漏的资料。检索覆盖官方产品文档、发布说明、固定版本源码、arXiv 预印本、Hacker News、Reddit、Cursor 论坛和中文社区。浏览统一使用 ego lite；官方仓库另用 GitHub 只读 API 核对提交、合并和发布日。搜索结果只用于发现资料，进入结论的来源均打开了原文或固定源码。

“新”有四种不同含义：首次发布、已有材料修订、旧稿漏收、本轮确认旧稿错误。本报告将它们分别标注。比如 Agents API 的公测日期有正式 changelog 支持；Microsoft 页面的 9 月 15 日只是文档更新时间；ACP v2 公告在 7 月，不能因为这次才查到就写成 9 月新协议。[1][8][16]

已有五个主案例仍可保留。新增材料有的适合补入产品接入表，有的更适合放在上下文、评测和进化章节。这里的“业界更新”不是产品排行榜，也不声称穷尽所有网站或私有社区。具体查询、访问时间、页面正文哈希、短证据和来源版次见同目录的 retrieval/、sources.jsonl、evidence.jsonl 及 run_manifest.json。

Main Analysis - 八项会影响书稿的变化

企业平台的采购对象开始拆成三个部分

OpenAI 在 9 月 10 日将 Agents API 作为公测发布。官方文档把会话编排、上下文压缩和恢复交给托管 Codex Harness；应用提供工具，并选择 OpenAI 托管或自托管执行环境。这比第十四章现有的 exec、SDK、App Server 三种接入面多了一个重要选项：平台可以不运行完整的本地 Harness 进程，而通过服务管理长会话。[1][2]

但“执行放在自己的机器上”不能改写成“所有数据都留在自己机器上”。截至本次访问，Agents API 文档明确它只支持美国数据驻留，不支持 Zero Data Retention，选择自托管 sandbox 也不会改变这项限制。企业比较方案时需要同时画出控制流、工具输出流和持久会话存储位置。[2]

Cursor 9月2日的自管机器说明给出了同一种边界的另一实现：worker在企业机器上编辑文件、运行命令，通过出站HTTPS通道与云端连接；推理和规划留在Cursor云端，工具输出可能包含代码并回传，转录也可能在云端处理和存储。因此，第十五章需要区分云端托管VM、自管worker以及完整自部署三种情况，不能只用“云Agent”概括。[5]

Anthropic 4月的Managed Agents文章是上轮漏收的重要架构材料。它把持久会话、模型循环和执行沙箱分开，分别恢复、替换和扩展。9月的权限与压缩接口则是这条路线的近期延伸。[3][4]

建议在第十八、二十六、二十九章加入一个共同的责任表：谁运行循环、谁持久化会话、谁保管凭证、谁执行工具、谁作最终业务验收。这个表应随具体产品版本填写，不用“托管/开源”两个标签替代。以上三个供应商案例相互印证的是分离责任的趋势，不是它们已具有等同的恢复语义或合规能力。[2][4][5]

长会话的关键变成“运行中可调整”，而不只是恢复旧对话

OpenAI 9月3日的API更新包含异步工具调用、执行中的指令调整，以及保持缓存前缀的推理强度调整。它们改变了第六、七章需要讨论的事件：外部输入可能在一次响应尚未结束时到达，工具结果也可能分批返回。应用应定义取消、纠正、迟到结果和新旧指令版本如何排序，而不是假定严格的用户消息-模型响应交替。[1]

Anthropic 9月14日增加显式请求会话压缩的beta能力，返回签名的compaction block；9月10日Managed Agents增加auto权限策略及逐调用的evaluation记录。签名能提供供应商定义的内容完整性，但不能证明压缩摘要在业务语义上没有漏掉约束；自动权限判断也不等于企业风险负责人已经授权一切动作。[3]

ACP v2草案更直接地指出：prompt请求的响应不应拥有整段工作的生命周期。session可以继续产生事件，输入入队、调整指令和后台工作需要独立表达；消息与工具内容通过稳定ID更新。它是7月已出现的设计方向，仍需版本协商和功能开关，不能当作已经稳定部署的全行业协议。[16]

这使原书区分task、attempt、session、turn的方向更重要，但也要求伪代码真的落实区别。“本轮输出结束”“取消请求已接收”“外部提交已确认”和“业务验收已通过”应是不同事件。尤其不能在正文强调这些区别，却在状态图里把effect confirmed直接连到最终成功。

建议第六章补一条具体时间线：任务执行期间收到约束修订，旧工具调用仍在途，结果晚到；系统如何记录旧结果、阻止失效授权下的新动作、保留已产生副作用，并用新契约开始后续attempt。这个例子比再加几个状态名更能说明问题。[1][3][16]

子代理应按工作关系选择上下文，框架也没有退出舞台

LangChain 9月8日公开了Deep Agents子代理的两种上下文模式。isolated从单独任务说明开始；fork继承父代理状态，并处理末尾委派调用，使子代理接着已有调查工作往下做。官方给出的实践区分很有用：修复执行者通常需要已有上下文；独立审阅者则应该避免继承父代理对结果的判断。[7]

因此，“子代理用新上下文，所以能兼顾能力与成本”需要改成条件判断。独立研究可能适合隔离，但刚调查过的问题再交给完全空白的执行者，会重复搜索；反过来，把全部父轨迹交给审阅者，又可能带来锚定与敏感信息传播。父窗口缩小、全系统token减少和结果更可靠是三个不同指标。[7]

Microsoft Harness文档展示了另一条可组合路线：沿用Agent Framework的聊天客户端、上下文提供者、会话和中间件，组合历史持久化、任务清单、模式、审批、观测及可选有界循环。它的意义是把常见运行能力做成可配置组合，不是宣布所有任务都应使用同一套最复杂配置。[8]

LangSmith Engine的5月工程文也值得补回。它从简化轨迹筛出可疑样本，再深入调查，产出问题、评测器和回归样本，必要时交给独立修复代理。这里既有框架复用，也有明确的职责和可审查产物。[17]

本书应继续保留“LangChain没有简单消失”的历史说明，并把LangGraph、Deep Agents以及Microsoft Harness放入当代架构比较的旁栏。没有必要把五个主案例扩成十几个同样篇幅的产品介绍；需要补的是“成品工具、可嵌入运行时、组合框架、托管Harness”这几个采购与设计层次。[7][8][17]

自我进化开始被拆成可测任务，但远没有一个普遍有效的循环

HarnessDev把待评估对象从最终答案换成可运行的基础设施，分别测试从弱种子创建Harness，以及根据执行反馈修订已有Harness。论文发现模型可以造出有用系统，但在代码和搜索任务上仍与成熟参考实现有差距，演化收益并不稳定，换执行模型也会改变结果。更关键的是，Evolution每个创建模型-执行模型组合只有一条轨迹，最终未见任务评估又只覆盖SWE-Pro，不能由此证明长期泛化已解决。[9]

HarnessEvolve补的是诊断信号：给执行代理正确答案，生成经过检查的参考轨迹，再和失败路径比较、聚类错误，提出跨prompt、skill、工具和执行逻辑的修改。质量门和性能门用于筛选候选。这为第二十二章提供了具体机制，但必须说清参考答案从哪里来；成功路径也不一定是唯一正确路径，第一处差异只能作为诊断线索，不能自动成为因果根因。[12]

JIT-Agent则把生成Harness做成可训练能力。它使用固定四模块协议，并训练辅助模型来生成、修复和改进适合当前任务的Harness。这里“任务执行模型冻结”和“系统没有训练”不是一回事。本书四层分类可以保留，但应把它写成四类可变对象，允许一次方案同时训练生成器、更新运行策略与积累经验，而不是要求每项工作只能落在一层。[11]

9月10日的Ecdysis通过跨任务失败聚合和多角色诊断，试图区分系统性Harness缺陷与对特定模型弱点的迎合。它使用“training”一词，但所述优化阶段固定任务模型参数与环境，更新的是Harness。其接受条件主要围绕训练集整体分数，不能直接等同本书要求的独立泛化、非劣和安全放行。[21]

9月10日首发、15日修订的RSI路线图按改进过程的自主性分级。它和本书按修改对象划分的四层不是互斥理论：前者问“谁决定怎样改进和收集经验”，后者问“改了什么”。可以在第十九章用一张交叉表讲清，展望中保留递归改进方向，但不要论文标题暗示人类已退出改进回路。[19]

综合来看，自我进化章节最需要增加的是实验合同：冻结哪些组件、哪些数据能被看到、谁生成反馈、谁选择候选、终测用几次、训练/搜索费用怎么计算，以及换模型后是否重新验收。新论文提供了多种构造方法，同时也说明不能用一张平均分表替代这些问题。[9][11][12][21]

评测正在从一个总分拆成机制、过程与最终结果

9月17日的Harness组件实证研究固定执行循环，改变规划、动作接口和上下文管理，形成176个配对设置。它发现上下文管理的价值随窗口收紧而增加；工具接口的取舍与模型使用shell的能力有关。规划对弱模型可能是准确率支撑，对强模型则可能主要降低重复操作。它没有证明“只留bash永远最好”，因为实验只用了三个尺度的Nemotron和一个Mistral，规划与动作接口消融又只在一个上下文设置下进行。[13]

Google 9月9日的工程文强调行为评测：除了最终是否完成，还检查是否搜索实时事实、是否运行验证器、是否执行了预期的中间动作。文章也明确，复杂任务可以有多个正确路径，不能把固定工具序列当成唯一答案；单次模型运行有噪声，应观察批次结果，而且行为评测不能替代端到端评测。[30]

这与本轮书稿验证直接相关。JSON能解析，只能说明语法有效；schema能接受数据，只能说明结构满足约束；调用了测试工具，仍不证明测试针对封存的候选；原子claim都标supported，也不证明引文真的支持每个分句。应把这些结果分别命名，不再合成一个没有分母的“全部通过”。

书中已有的消融、配对实验、回归与独立验收思路值得保留，但需要可计算定义。 2×2 设计中的一个格子是一个条件均值，模型主效应、Harness主效应与交互项需要比较多个格子；“移除组件后掉分”是该实验条件下的证据，不是普遍必要性的证明。统计不显著也不等于非劣，零观测违规也不等于风险为零。

建议第十二章分成三种清晰结果：契约/状态机检查、模型行为评测、业务终态评测。第二十四章另给候选选型与终测的数据隔离、重复试验和停止规则。这样的分工既吸收了最新工程实践，也能减少目前第十、十二、十九和二十四章的重复。[9][13][30]

长时自治与效率优化成为单独目标，必须报告代价

Harness-of-Harness在现成coding harness外组织规划、实现、测试循环，要求小而可验证的增量，并将实现时测试与独立验收分离。它报告三轮后的相对增益，也提供同开发轮数的对照。但不同轮数、token、时间和评估费用仍需分别看，不能把“多跑了三轮的产物更完整”直接归因为某一个编排组件的效果。[10]

SoL-Pi在大量环境回路中搜索后保留四种机制，涉及动作合并、压缩时机、观察结果处理和委派读取。它的具体工程启发在于：减少模型往返，比较缓存重写费用与未来节省，把长日志中的证据保留下来。但效率配置在GPT-5.6 Sol的EdgeBench结果中，从Pi的44.8分变为42.0分，保留93.7%的分数；这是质量-成本折中，而非严格无损。在Terminal-Bench的63个CPU任务上，文中Pi完成18项、SoL-Pi完成15项，同样不能只报更低费用。[23]

Stellar Colosseum把长证明分为相互依赖的部分，先探索候选路线，再用针对性的反证和验证结果修复相关部分。它扩展了本书“委派-合并-重验”的例子：在研究任务中，需要检查结论间的依赖是否一起成立，而不是把独立段落简单拼接。论文中的研究成果和评测数字是作者报告，本轮没有复现证明或重跑大规模多Agent实验。[22]

Cursor Projects则提供产品侧的对应实践：协调者委派、跨会话共享文件、事件触发与周期执行。产品所说的长期运行能力，仍需要企业自己定义预算、失败后停止条件、来源污染处理和提交权限。本书不宜把“能够持续运行”写成“长期任务可靠性已经解决”。[6]

建议第十一、二十、二十二章统一费用口径：任务执行费用、候选搜索费用、评审/验证费用、失败重试费用和人工接管费用分别记录。一次探索产生的模型调用越多，越需要说明这份成本如何被后续复用摊薄。平均每小时省多少钱，只有在任务分布、利用率与计价条件明确时才有可迁移含义。[9][10][23]

安全边界必须覆盖“运行时自己执行的配置”与持久记忆

HookPry研究的是生命周期hook的更新通道：已获信任的插件更新后，新增或改变事件绑定的命令，可能在模型看不见的时候由宿主执行。其威胁模型要求攻击者控制插件元数据、版本和hook配置，并依赖更新被采用及相应事件发生。它不意味着任意网页都能绕过全部沙箱，也不能据论文实验断言所有现有版本仍有同一漏洞。[14]

对本书而言，关键补充不是再说一次“警惕提示注入”，而是承认有些危险动作根本不经模型提议工具的那条路径。第九、十三、十六章应增加hook清单差异、更新后权限重新确认、执行身份、宿主与工作区隔离、审计记录及紧急撤销。把hook作为唯一安全边界会遗漏hook本身的执行权。[14]

MemSecBench将记忆写入、后续任务引用、产生行为后果以及选择性删除放在同一组案例里考察。它是7月旧材料，但能补强第二十一章目前偏重“如何积累经验”的叙述：入库检查通过不等于未来读取安全；删除一条记忆也不等于其摘要、索引、派生skill或下游产物里的影响已消失。[15]

社区资料可帮助设计回归场景。Cursor论坛的具体报告描述子代理没有按角色卡回应，以及此前出现过递归派生；发帖者并没有证明二者有相同根因。应把它转成待验证问题，例如角色身份是否只靠prompt、嵌套深度是否由外部控制、取消是否覆盖后代，不能据此宣布产品已发生权限穿透。[34]

建议在第九、二十一、二十四章加入一个完整的“坏经验如何进入、何时被读到、如何撤回”的案例。测试应检查撤销后已有会话、缓存、检索索引和已派生能力的行为。安全评测的结果按攻击前提和版本报告，不能用单一成功率替代信任边界说明。[14][15]

新接口会延伸到更多领域，但旧证据也需要重新绑定版本

HEART把LLM包装在工具接口里，让外层代理通过更自然的输入完成多步工具组合。这给第八章提供一个不同于直接暴露全部schema的选择，但它只是把模式解析和失败处理移到包装内，增加了一层模型调用和归因问题。不能把“外层看不到schema”解释为不再需要精确参数、权限检查或可审计结果。[18]

Show-Harness用语义动作接口连接VLM与具体机器人解释器；openJiuwen则强调结构可组合与运行时适应两个方面。这些材料适合第三十章的展望或原理章的短反例，不必把以企业软件 and 知识任务为主的书突然变成机器人教材。尤其物理动作有不同的实时性、安全和验证要求，本文没有实际运行机器人。[20][31]

与此同时，本轮回查发现原书引用版次有错误。GSME名称和v1文稿对应，但登记表把它记为v2；v2已经使用HarnessBank名称。相关机制有延续，不能因此把整篇研究说成无效，但书名、作者、版次、短证据和机制描述应统一到选定版本，再核对差异。[26][27]

Self-Harness仍是8月20日的v3，Living-Harness仍是8月11日的v2，本轮没有发现新的版本。NLE官方资料确认全称是NetHack Learning Environment，HSI也把它和MiniHack作为复杂roguelike环境讨论；正文“非语言增强任务”是确实需要勘误的解释。[24][25][28][29]

还有一篇十一系统源码综述值得追踪，但本次所见2609.00006页面的编号月份和提交日期不一致。它的检索信息不应直接被归为9月新发布。即使其特定代码样本中未发现通用Agent框架或向量检索，也只能描述那份样本，不能推断整个行业都没有这些实现。同期组件实证研究就明确使用LangGraph。[13][36]

Synthesis - 对全书主线的判断

全书最有价值的主线仍然成立：模型输出需要经过工具、状态、权限、反馈和验收，才能变成可靠工作。新材料强化了三件事：模型循环与执行环境可分别管理；改进对象可以是运行基础设施本身；质量、成本和风险必须一起衡量。[1][4][9][23]

但本书目前常把一个合理工程方向写成普遍要求。例如“必须外置”“必然更好”“能够证明”“不可变治理平面”等表述，需要补出任务风险、模型能力、成本和威胁模型这些条件。教学用简化是必要的，简化后的伪代码却仍须保持正文声称的关键不变量，尤其拒绝就不能执行、未知提交不能盲重试、提交成功不能自动代表业务完成。

四层进化最好保留为“本次改了什么”的分类，同时增加两条独立轴：改进决策由谁控制，以及效果用什么独立证据确认。这样既能容纳JIT-Agent训练生成器的混合方案，也不会把RSI自主性路线图误当成从记忆到模型的固定升级阶梯。[11][19]

Counterevidence Register - 反证与限制

容易写过头的结论	本轮证据要求的收窄
更复杂的Harness更强	组件消融依赖模型、预算和任务，不能按模块数排名。[13]
自我进化会持续增长	HarnessDev显示轨迹波动和有限迁移，Evolution重复次数有限。[9]
省token而质量不变	SoL-Pi效率配置有明确得分让步；需写非劣界和置信区间。[23]
自托管就不向外传数据	执行环境位置、推理位置、持久存储和ZDR是不同问题。[2][5]
参考轨迹能识别真正根因	需要正确答案，合法路径可能不唯一，诊断还依赖评估代理。[12][35]
社区好评证明谁最强	Reddit任务集小且自报配置，HN存在分歧，不能当同条件榜单。[32][33]
有权限hook即可安全	hook更新及宿主执行本身也是攻击面。[14]
metadata verified等于事实已核查	当前引用脚本核查关系和字段，版次误配仍可能通过；GSME就是具体反例。[26][27]

Claims-Evidence Table - 关键结论如何复核

结论	直接证据	对应书稿
新增托管Codex Harness接入面	API changelog的Sep 10条目与overview；两者来自同一供应商，不虚报为独立来源簇。[1][2]	14、18、29
执行环境与Harness管理应拆开	OpenAI overview、Anthropic架构、Cursor自管机器三组官方资料。[2][4][5]	5、9、26
子代理需要选择上下文模式	Deep Agents context modes原文；Google评测文和HarnessDev分别补充行为/泛化验收边界。[7][9][30]	7、11、12
进化不能只看开发集上涨	HarnessDev、HarnessEvolve、SoL-Pi的不同方法与限制。[9][12][23]	19-24
当前框架能力仍在扩展	Microsoft Harness、Deep Agents、LangSmith Engine。[7][8][17]	2、18
持久配置和记忆需要独立治理	HookPry、MemSecBench，以及原书本轮安全伪代码负例。[14][15]	9、21、24
原书需要版次/译名勘误	GSME v1/v2页面和NLE官方README。[26][27][28]	19、22、附录C

Limitations - 本轮能确认到哪里

检索完成了有边界的广泛覆盖，没有办法证明已经“找遍全网”。登录墙、未索引页面、私有社区、当天稍后出现的更新及厂商未公开实现都不在确定覆盖范围。中文二手文章帮助发现方向，关键技术结论回到论文、文档或代码，不按转载次数提高可信度。

论文大多是预印本。报告中的实验结论是作者在其任务、模型、费用口径下的结果，本轮只复核方法、结果和限制，没有重跑大规模训练、供应商完整业务流程、付费模型矩阵或机器人实验。书中案例的本地执行另有报告，不能把它替代论文复现。

来源记录的“supported_as_scoped”只表示精确限定的来源主张有对应短证据。网页访问成功、元数据匹配和代码能执行，均不自动升级为生产可靠性证明。36项研究来源与官方仓库探针的原始证据分开保存，查询页面与同一论文的摘要/全文不重复算作独立证据簇。本轮另补查了BigQuery原方言：原书的时间旅行语法有官方依据，但查数周前的字面量需要考虑保留期，长久复现应保留物化快照或版本化数据。SQLite不支持这段语法本身不是证明原查询写错；原引擎执行未验证。[37]

Recommendations - 建议如何修改

第一步处理正确性：修补拒绝分支和未知副作用语义；改正ACI、NLE、validation等误译；修正Codex协议名和dsh虚拟机表述；把第25章补丁/SQL和第27章反例变成能够复现的成套案例。这些是已有内容的问题，不应等待新章节写完才处理。

第二步补上当前架构：第十三至十八章按具体产品面与版本重新组织，加入托管Harness、自管worker、新OpenHands SDK和上下文模式。五个主案例保留，其他框架放在对照表或短旁栏；避免用未经同条件测试的“成熟”“成本高”作客观排名。

第三步重写进化章节的实验说明：将HarnessDev、HarnessEvolve、JIT-Agent、Ecdysis和SoL-Pi放进各自机制的位置，注明数据用途、统计单位、选择偏差、费用和模型迁移。把“进化成立”从一句结论变成读者能运行的判定流程。

最后才做文体编辑和发布。原则是先给情境，再说动作及原因，最后补术语；尽量统一中文名，将英文原名集中在首次出现和术语表。机械守恒、语义复核、案例执行和发布构建分别出结果，旧审计的错误通过公开勘误更正，而不是静默覆盖历史。

Methodology - 方法与复核入口

共保留20条主题检索和2条SQL方言补充查询快照；按正式发布说明、论文submission history、文档更新时间与GitHub固定commit区分日期。浏览任务共用ego lite空间25，检索与正文在读取后持久化；微软页面曾误取推广卡片，发现后已改为提取main正文并重新保存。

[compile_research.py](#)逐项验证短引文在捕获正文中存在，并复算正文SHA-256，再输出37条来源、37条证据和37条限定主张（36项研究资料，加1项案例方言资料）。该脚本不从URL或标题自动推导真实性。两个独立文稿审阅与一个新论文方法复核报告提供交叉意见，主代理再以原文和运行结果校准结论；它们不是外部人类同行盲审。

另见[review_history_principles_style.md](#)、[review_products_evolution_structure.md](#)、[review_new_paper_methods.md](#)、[report_vendor_deltas.md](#)和[report_examples_execution.md](#)。示例执行的通过与失败以原始JSON、进程退出码和运行日志为准。

Bibliography - 参考资料

- [1] OpenAI (2026). "Changelog". 2026-09-10. <https://developers.openai.com/api/docs/changelog> (Retrieved: 2026-09-19).
- [2] OpenAI (2026). "Agents API". snapshot 2026-09-19. <https://developers.openai.com/api/docs/guides/agents-api/overview> (Retrieved: 2026-09-19).
- [3] Anthropic (2026). "Claude Platform release notes - Claude Platform Docs". 2026-09-03/10/14. <https://docs.anthropic.com/en/release-notes/api> (Retrieved: 2026-09-19).
- [4] Anthropic (2026). "Scaling Managed Agents: Decoupling the brain from the hands \ Anthropic". 2026-04-08. <https://www.anthropic.com/engineering/managed-agents> (Retrieved: 2026-09-19).
- [5] Cursor (2026). "在您自行管理的机器上运行云端智能体 · Cursor". 2026-09-02. <https://cursor.com/blog/self-hosted-machines> (Retrieved: 2026-09-19).
- [6] Cursor (2026). "Cursor 最新动态 - 最新更新与发布说明". 2026-09-10. <https://cursor.com/changelog> (Retrieved: 2026-09-19).
- [7] T. Bengre; C. Curme / LangChain (2026). "Organizing Context in a Multi-Agent Harness". 2026-09-08. <https://www.langchain.com/blog/organizing-context-in-a-multi-agent-harness> (Retrieved: 2026-09-19).
- [8] Microsoft (2026). "Agent Harness | Microsoft Learn". updated 2026-09-15. <https://learn.microsoft.com/en-us/agent-framework/concepts/harness> (Retrieved: 2026-09-19).
- [9] Wu et al. (2026). "HarnessDev: Can LLMs Create and Evolve Their Own Agent Harness?". 2609.01437v1; 2026-09-01. <https://arxiv.org/html/2609.01437v1> (Retrieved: 2026-09-19).
- [10] Yan et al. (2026). "Harness-of-Harness: Multi-Day Autonomous Software Development with Continual Improvement". 2609.01481v1; 2026-09-01. <https://arxiv.org/html/2609.01481v1> (Retrieved: 2026-09-19).
- [11] Zhang et al. (2026). "JIT-Agent: Scaling Harness Intelligence via Just-in-Time Harness Evolution". 2608.25593v2; 2026-09-03. <https://arxiv.org/html/2608.25593v2> (Retrieved: 2026-09-19).
- [12] Jiang et al. (2026). "HarnessEvolve: Learning from Reference Trajectories for Reliable Agent Self-Evolution". 2609.00829v1; 2026-09-01. <https://arxiv.org/html/2609.00829v1> (Retrieved: 2026-09-19).
- [13] Fan et al. (2026). "An Empirical Study of Harness Design for Coding Agents". 2609.20804v1; 2026-09-17. <https://arxiv.org/html/2609.20804v1> (Retrieved: 2026-09-19).

- [14] Li et al. (2026). "A Blind Trust, the Bloody Thrust: When Attacker-Controlled Hook Updates Steer AI Agent Harnesses towards Malicious Behaviors". 2609.03884v2; 2026-09-08. <https://arxiv.org/html/2609.03884v2> (Retrieved: 2026-09-19).
- [15] Chen et al. (2026). "MemSecBench: Tracking Agent Memory Poisoning from Persistence to Consequence and Repair". 2607.27080v1; 2026-07-29. <https://arxiv.org/abs/2607.27080> (Retrieved: 2026-09-19).
- [16] Ben Brandt / ACP (2026). "ACP v2 is available in Draft - Agent Client Protocol". 2026-07-20 draft. <https://agentclientprotocol.com/announcements/acp-v2-draft> (Retrieved: 2026-09-19).
- [17] Palash Shah / LangChain (2026). "LangSmith Engine: How We Built an Agent for Improving Agents". 2026-05-19. <https://www.langchain.com/blog/how-we-built-langsmith-engine-our-agent-for-improving-agents> (Retrieved: 2026-09-19).
- [18] Jin et al. (2026). "Harness Engineering in LLM Tool Use via Agent-Native Reusable Tool Primitives". 2609.01736v1; 2026-09-01. <https://arxiv.org/abs/2609.01736> (Retrieved: 2026-09-19).
- [19] Duan et al. (2026). "The Last AI Built by Humans: Toward Genuine Recursive Self-Improvement". 2609.11873v2; 2026-09-15. <https://arxiv.org/html/2609.11873v2> (Retrieved: 2026-09-19).
- [20] Chen et al. (2026). "Show-Harness: Just a VLM Agent Can Play Robots". 2609.10522v1; 2026-09-09. <https://arxiv.org/abs/2609.10522> (Retrieved: 2026-09-19).
- [21] Yue et al. (2026). "Ecdysis: Efficient and Effective Training of Runtime Harnesses for LLM Agents". 2609.11677v1; 2026-09-10. <https://arxiv.org/html/2609.11677v1> (Retrieved: 2026-09-19).
- [22] Lin et al. (2026). "Stellar Colosseum: A Many-Agent Harness for Long-Horizon Research in Mathematics and Theoretical Computer Science". 2609.15983v2; 2026-09-15. <https://arxiv.org/html/2609.15983v2> (Retrieved: 2026-09-19).
- [23] Liu et al. (2026). "SoL-Pi: Recursively Scaling Auto-Research Loops for Efficient Agent Harness". 2609.20519v1; 2026-09-17. <https://arxiv.org/html/2609.20519v1> (Retrieved: 2026-09-19).
- [24] Zhang et al. (2026). "Self-Harness: Harnesses That Improve Themselves". 2606.09498v3; 2026-08-20. <https://arxiv.org/abs/2606.09498> (Retrieved: 2026-09-19).
- [25] Du et al. (2026). "Living-Harness Is an Interactive-Agent Evolver". 2607.26598v2; 2026-08-11. <https://arxiv.org/abs/2607.26598> (Retrieved: 2026-09-19).
- [26] Luo et al. (2026). "Self-Evolving Agent Harnesses via Gated Semantic Quality-Diversity". 2607.13683v1; 2026-07-15. <https://arxiv.org/abs/2607.13683v1> (Retrieved: 2026-09-19).
- [27] Luo et al. (2026). "HarnessBank: Semantic Gene-Bank Search with Gated Verification for Agent-Harness Self-Evolution". 2607.13683v2; 2026-07-30. <https://arxiv.org/html/2607.13683v2> (Retrieved: 2026-09-19).
- [28] NLE authors / Facebook Research (2026). "facebookresearch/nle: The NetHack Learning Environment". snapshot 2026-09-19. <https://github.com/facebookresearch/nle> (Retrieved: 2026-09-19).
- [29] Tailin Zhou (2026). "Hierarchical Self-Improvement: A Framework for Task-Specific Evolvable Agent Harnesses". 2608.08466v1. <https://arxiv.org/html/2608.08466v1> (Retrieved: 2026-09-19).
- [30] Taylor Mullen; Christian Gunderman / Google (2026). "The Anatomy of Harness Engineering: How to Evaluate, Iterate, and Guard AI Coding Agents - Google Developers Blog". 2026-09-09. <https://developers.googleblog.com/the-anatomy-of-harness-engineering-how-to-evaluate-iterate-and-guard-ai-coding-agents/> (Retrieved: 2026-09-19).
- [31] openJiuwen Team (2026). "openJiuwen: Beyond Static Harnesses for Long-Horizon Coding Agents". 2608.27969v1; 2026-08-28. <https://arxiv.org/abs/2608.27969> (Retrieved: 2026-09-19).

- [32] Hacker News participants (2026). "An empirical study of harness design for coding agents | Hacker News". 2026-09-18/19. <https://news.ycombinator.com/item?id=49753878> (Retrieved: 2026-09-19).
- [33] r/LocalLLaMA participants (2026). "Which agent harness do you use and why? : r/LocalLLaMA". 2026-09-05. https://www.reddit.com/r/LocalLLaMA/comments/1w8f7bp/which_agent_harness_do_you_use_and_why/ (Retrieved: 2026-09-19).
- [34] Cursor forum reporter TeX1 (2026). "Custom subagent ignores its card and claims it is the parent agent - production unusable - Support / Bug Reports - Cursor - Community Forum". 2026-09-17. <https://forum.cursor.com/t/custom-subagent-ignores-its-card-and-claims-it-is-the-parent-agent-production-unusable/172038> (Retrieved: 2026-09-19).
- [35] 黄浴 / 知乎 (2026). "HarnessEvolve：从参考轨迹中学习以实现可靠的智能体自进化 - 知乎". 2026-09-06. <https://zhuanlan.zhihu.com/p/2079901486545478621> (Retrieved: 2026-09-19).
- [36] Barbaste et al. (2026). "Harness Engineering: Anatomy, Architecture, and Evolution of Coding Agents -- A Source-Code Study of Eleven Systems". 2609.00006v1; page date conflicts with identifier month. <https://arxiv.org/abs/2609.00006> (Retrieved: 2026-09-19).
- [37] Google Cloud (2026). "Query syntax | BigQuery". snapshot 2026-09-19. https://docs.cloud.google.com/bigquery/docs/reference/standard-sql/query-syntax#for_system_time_as_of (Retrieved: 2026-09-19).